PRODUCT REQUIREMENTS DOCUMENT

Módulo de Inteligência Eleitoral

VereadorCRM — Painel de Dados do TSE

Versão	1.0
Status	Draft — para revisão
Produto	VereadorCRM
Módulo	Inteligência Eleitoral (/eleicoes)
Stack	PostgreSQL 17 · Redis 7 · Node.js · React · Evolution API
Infraestrutura	Docker Compose (5 serviços) — crmvere.com.br
Data	Abril 2025
Autor	Paulo — Product Owner

© 2025 VereadorCRM

1. Visão Geral

1.1 Objetivo

O Módulo de Inteligência Eleitoral adiciona ao VereadorCRM um painel dedicado (/eleicoes) que importa e processa dados públicos do TSE, cruza com o cadastro do vereador no sistema e entrega inteligência territorial acionável: onde o vereador foi votado, em quais escolas, em quais bairros, e como isso evoluiu entre eleições.

1.2 Problema resolvido

Vereadores e assessores não têm hoje uma forma rápida e não-técnica de entender a base eleitoral por território. Os dados existem — são públicos no TSE — mas estão em CSVs brutos com milhões de linhas. O módulo transforma esses dados em inteligência prática dentro do mesmo CRM que o vereador já usa para gerenciar constituintes.

1.3 Stack do sistema atual

O VereadorCRM roda em Docker Compose com os seguintes serviços:

Serviço	Container	Função
`db`	`vereador_crm_db`	PostgreSQL 17-alpine — banco principal multi-tenant
`redis`	`vereador_crm_redis`	Redis 7 — cache de sessão e filas de jobs
`evolution_api`	`vereador_crm_whatsapp`	Evolution API — gateway WhatsApp (wa.crmvere.com.br)
`backend`	`vereador_crm_backend`	Node.js na porta 3001 — API REST (api.crmvere.com.br)
`frontend`	`vereador_crm_frontend`	React servido via Nginx na porta 3000 (crmvere.com.br)

Proposta de valor

O vereador abre /eleicoes e vê: onde foi mais votado, em quais escolas perdeu terreno entre 2020 e 2024, quais bairros têm alto eleitorado com baixa penetração (oportunidade), e como sua base territorial se cruza com os constituintes já cadastrados no CRM — tudo sem sair do sistema.

2. Arquitetura de Dados

2.1 Fontes — Portal de Dados Abertos do TSE

Todos os dados eleitorais são públicos (dadosabertos.tse.jus.br). Nenhuma API privada ou scraping é necessário.

Arquivo ZIP do TSE	Conteúdo	Usado para
`votacao_secao_{ano}_{UF}.zip`	Votos por candidato/seção	Calcular votos por escola e bairro
`perfil_eleitorado_local_votacao_{ano}.zip`	Nome, endereço e bairro de cada local	Cruzar seção → escola → bairro
`perfil_eleitorado_secao_{ano}.zip`	Perfil demográfico por seção	Dados de eleitorado por local
`candidatos_{ano}_{UF}.zip`	Nome, número, partido, situação	Identificar o candidato cadastrado

2.2 Script de importação (Python)

Um script Python standalone — independente do backend Node.js — é responsável por baixar, processar e inserir os dados no banco. Ele se conecta diretamente ao PostgreSQL via DATABASE_URL do .env.

```
# tse_import.py — roda fora do docker ou como container separado
import pandas as pd, psycopg2, requests, zipfile, os

ANO       = 2024
UF        = 'PR'
MUNICIPIO = 'APUCARANA'
DB_URL    = os.getenv('DATABASE_URL')

# 1. Baixa ZIPs do TSE
# 2. Filtra por município e cargo (13 = vereador)
# 3. Cruza votacao_secao + local_votacao por nr_zona + nr_local_votacao
# 4. Insere em tse_votos_secao, tse_locais_votacao, tse_candidatos
# 5. Atualiza jobs.status = 'done' no Redis
```

2.3 Schema no PostgreSQL (banco vereador_crm)

As tabelas abaixo são criadas no schema public do PostgreSQL já existente no container vereador_crm_db. Row Level Security é implementada via coluna tenant_id, consistente com o restante do sistema.

tse_candidatos

Coluna	Tipo	Descrição
`id`	SERIAL PK	Chave primária
`tenant_id`	UUID FK	Referência ao tenant (vereador) no sistema
`ano_eleicao`	INTEGER	Ano da eleição (2020, 2024...)
`nm_candidato`	TEXT	Nome completo conforme TSE
`nr_candidato`	INTEGER	Número na urna
`sg_partido`	TEXT	Sigla do partido
`cd_municipio`	INTEGER	Código TSE do município
`nm_municipio`	TEXT	Nome do município
`ds_situacao`	TEXT	Eleito / Não eleito / Suplente
`qt_votos_total`	INTEGER	Total de votos na eleição (calculado)
`created_at`	TIMESTAMPTZ	Data de importação

tse_locais_votacao

Coluna	Tipo	Descrição
`id`	SERIAL PK	Chave primária
`ano_eleicao`	INTEGER	Ano de referência
`cd_municipio`	INTEGER	Código do município
`nr_zona`	INTEGER	Zona eleitoral
`nr_local_votacao`	INTEGER	ID do local de votação
`nm_local_votacao`	TEXT	Nome da escola / ginásio
`ds_endereco`	TEXT	Endereço completo
`nm_bairro`	TEXT	Bairro

Coluna	Tipo	Descrição
nr_cep	TEXT	CEP
qt_eleitores	INTEGER	Total de eleitores cadastrados no local

tse_votos_secao

Coluna	Tipo	Descrição
id	SERIAL PK	Chave primária
ano_eleicao	INTEGER	Ano da eleição
cd_municipio	INTEGER	Código do município
nr_zona	INTEGER	Zona eleitoral
nr_secao	INTEGER	Seção eleitoral
nr_local_votacao	INTEGER	ID do local (FK para tse_locais_votacao)
nr_candidato	INTEGER	Número do candidato
qt_votos	INTEGER	Votos nessa seção

2.4 View materializada — votos_por_bairro

Para performance no frontend, criar uma view materializada que pré-agrega os dados. Atualizar após cada importação.

```
CREATE MATERIALIZED VIEW votos_por_bairro AS
SELECT
  v.ano_eleicao,
  c.tenant_id,
  c.nm_candidato,
  c.sg_partido,
  l.nm_bairro,
  l.nm_local_votacao,
  l.ds_endereco,
  l.qt_eleitores,
  SUM(v.qt_votos) AS qt_votos,
  ROUND(SUM(v.qt_votos)::numeric / NULLIF(l.qt_eleitores,0) * 100, 1) AS
pct_eleitorado
FROM tse_votos_secao v
  JOIN tse_candidatos c    USING (ano_eleicao, cd_municipio, nr_candidato)
  JOIN tse_locais_votacao l USING (ano_eleicao, cd_municipio, nr_zona,
nr_local_votacao)
GROUP BY 1,2,3,4,5,6,7,8;
```

```
-- Índice para queries do frontend
CREATE INDEX ON votos_por_bairro (tenant_id, ano_eleicao, nm_bairro);
```

2.5 Job assíncrono via Redis

A importação TSE é demorada (3-10 min). O fluxo usa a fila Redis já existente no sistema para não bloquear o backend:

1. Frontend chama POST /api/eleicoes/importar com { ano, municipio, nr_candidato }
2. Backend publica job na fila Redis: queue:tse_import
3. Worker Node.js (ou script Python via exec) consome a fila e executa a importação
4. Worker atualiza chave Redis tse_import:{tenant_id}:{ano} com status e progresso
5. Frontend faz polling em GET /api/eleicoes/status e exibe barra de progresso
6. Ao concluir: worker dispara evento SSE ou webhook interno para atualizar a UI

Otimização: cache por município

Os dados brutos do TSE (tse_locais_votacao e tse_votos_secao) são iguais para todos os tenants do mesmo município. Importar uma vez e compartilhar. A view votos_por_bairro filtra por tenant_id — cada vereador vê apenas seus próprios dados.

3. Endpoints do Backend (Node.js — porta 3001)

Todos os endpoints abaixo são prefixados com /api/eleicoes e requerem JWT de autenticação (padrão do sistema). O backend acessa o PostgreSQL via DATABASE_URL e o Redis via REDIS_URL, ambos já disponíveis no container vereador_crm_backend.

Método + Rota	Descrição	Resposta
GET /resumo	Cards de alto nível da eleição selecionada	{ totalVotos, ranking, bairroTop, escolaTop, variacao }
GET /bairros	Tabela de votos por bairro com variação histórica	[{ nm_bairro, qt_votos, pct, variacao, escolas[] }]
GET /escolas	Ranking de locais de votação com dados completos	[{ nm_local, nm_bairro, ds_endereco, qt_votos, pct }]
GET /historico	Comparativo de votos entre eleições disponíveis	[{ ano, qt_votos, bairros[] }]
GET /insights	Análise automática: oportunidades, redutos, riscos	[{ tipo, nm_bairro, dado, sugestao }]
POST /importar	Inicia job de importação assíncrona dos dados TSE	{ jobId, status: 'queued' }
GET /status	Consulta progresso do job de importação	{ status, progresso, eta }
GET /exportar	Gera CSV para download dos dados do painel	arquivo .csv (stream)
GET /candidatos/busca?q=	Busca candidatos pelo nome para setup inicial	[{ nm_candidato, nr_candidato, sg_partido }]

4. Funcionalidades do Painel (/eleicoes)

4.1 Mapa de features — MVP

Funcionalidade	Descrição	Prioridade
Setup / Onboarding	Wizard para vincular candidato TSE ao tenant; disparar importação	**Must Have**
Barra de progresso	Feedback em tempo real do job de importação via polling Redis	**Must Have**
Aba Resumo	7 cards de métricas de alto nível com seletor de eleição	**Must Have**
Aba Bairros	Tabela interativa com ordenação, filtro e expansão por escola	**Must Have**
Aba Escolas	Ranking de locais com dados completos e link para Maps	**Must Have**
Aba Comparativo	Variação de votos entre eleições por bairro e escola	**Must Have**
Aba Insights	Análise automática: oportunidades, redutos e riscos territoriais	**Should Have**
Exportar CSV	Download dos dados filtrados em .csv	**Should Have**
Cruzar com CRM	Quantos constituintes do CRM estão em cada bairro vs votos recebidos	**Could Have**
Comparar concorrentes	Mesmo painel para outros vereadores eleitos do município	**Could Have**

4.2 Aba Resumo — Cards

Tela inicial. Seletor de eleição no topo (2024 | 2020 | 2016). Cards atualizam via query ao endpoint GET /resumo.

- Total de votos recebidos
- Posição no ranking de vereadores eleitos do município
- Número de bairros com ao menos 1 voto
- Escola com maior concentração de votos
- Bairro com maior concentração de votos
- Variação percentual em relação à eleição anterior

- % de penetração sobre o total de eleitores do município

4.3 Aba Bairros

Tabela interativa com dados de todos os bairros do município. Principal funcionalidade para planejamento territorial.

Colunas

- Bairro
- Votos recebidos
- % do total de votos do candidato
- Total de eleitores no bairro
- % de penetração (votos / eleitores)
- Variação vs eleição anterior (+/- votos e %)
- Nº de escolas no bairro

Comportamentos

- Ordenação por qualquer coluna
- Filtro textual por bairro
- Ao clicar em bairro: expande linha mostrando escolas com mesmas colunas
- Badge de tendência: seta verde (cresceu) / vermelha (caiu) / traço (estável)
- Barra proporcional ao % de penetração na coluna visual

4.4 Aba Escolas

Ranking dos locais de votação ordenados por votos recebidos. Permite precisão de endereço para planejamento de eventos.

- Posição, nome, bairro, endereço, votos, total eleitores, % penetração
- Filtro por bairro, eleição e busca por nome
- Ao clicar: card lateral com dados completos + botão 'Ver no Google Maps'
- Comparativo inline com eleição anterior

4.5 Aba Comparativo Histórico

- Gráfico de barras: votos totais por eleição disponível (2016, 2020, 2024)
- Tabela com colunas por eleição e delta absoluto/percentual para cada bairro
- Top 5 bairros que mais cresceram e top 5 que mais regrediram
- Indicador de escolas que mudaram de zona entre eleições

4.6 Aba Insights de Campanha

Análise automática gerada no endpoint GET /insights a partir dos dados da view materializada.

Tipos de insight gerados

7. Oportunidade — bairro com eleitorado alto e penetração abaixo de 5%
8. Reduto — bairro com penetração acima de 15% (manter e ativar base)
9. Risco — bairro com queda superior a 20% entre eleições
10. Terreno neutro — bairro sem nenhum voto registrado (novo território)

Cada insight exibe: nome do bairro, dado que suporta o insight, e texto de sugestão de ação. Exemplo: 'Jardim América tem 1.840 eleitores e você recebeu 38 votos (2,1%). Considere cadastrar demandas desse bairro no CRM ou realizar uma agenda regional.'

5. Fluxo de Setup do Módulo

5.1 Primeiro acesso

11. Vereador acessa /eleicoes pela primeira vez
12. Sistema verifica se existe registro em tse_candidatos para o tenant_id
13. Se não existe: exibe wizard de configuração (3 passos)
14. Se existe: exibe painel direto com dados da eleição mais recente

5.2 Wizard de configuração

Passo 1 — Identificar candidato

- Campo de busca: digitar nome → chama GET /candidatos/busca?q= → exibe lista filtrada
- Usuário seleciona o candidato correto (nome + número + partido)
- Município é herdado automaticamente do cadastro do tenant

Passo 2 — Selecionar eleições

- Checkboxes: 2024, 2020, 2016 (conforme disponibilidade no TSE para o município)
- Estimativa de tempo de importação exibida por eleição

Passo 3 — Confirmar e importar

- Resumo da configuração para revisão
- Botão 'Importar dados' dispara POST /importar
- Wizard fecha e painel exibe barra de progresso com status do job Redis

6. Integração com os Demais Módulos do VereadorCRM

6.1 Autenticação

O módulo reutiliza o JWT já emitido pelo backend Node.js. Nenhum mecanismo de auth adicional é necessário. O tenant_id extraído do token é usado como filtro em todas as queries do módulo eleitoral.

6.2 Constituintes (cruzamento — Could Have)

O banco já contém a tabela de constituintes com campo de bairro (ou endereço). A query de cruzamento junta essa tabela com a view votos_por_bairro pelo campo nm_bairro normalizado:

```
SELECT
  v.nm_bairro,
  v.qt_votos,
  COUNT(c.id) AS qt_constituintes_crm,
  v.pct_eleitorado
FROM votos_por_bairro v
  LEFT JOIN constituintes c
    ON UNACCENT(LOWER(c.bairro)) = UNACCENT(LOWER(v.nm_bairro))
    AND c.tenant_id = v.tenant_id
WHERE v.tenant_id = $1 AND v.ano_eleicao = $2
GROUP BY 1,2,4;
```

6.3 WhatsApp / Evolution API

Os dados do módulo eleitoral alimentam uma funcionalidade futura (v1.2): segmentação de envios em massa por bairro. O vereador seleciona um bairro no painel e dispara via Evolution API uma mensagem para todos os constituintes daquele bairro cadastrados no CRM. Isso conecta os dados de base eleitoral com a automação de WhatsApp já existente.

7. Infraestrutura e Deploy

7.1 Sem novos containers no Docker Compose

O módulo eleitoral NÃO requer nenhum novo serviço Docker. Toda a lógica se encaixa nos containers existentes:

Responsabilidade	Container	Como
Banco de dados TSE	`db` (`vereador_crm_db`	Novas tabelas no schema public do PostgreSQL existente
Fila de jobs	`redis` (`vereador_crm_redis`	Nova queue: tse_import no Redis já ativo
Worker de importação	`backend` (porta 3001	Novo worker Node.js que consome a fila e executa o script Python
Endpoints da API	`backend` (porta 3001	Novo router /api/eleicoes no Express existente
Frontend do painel	`frontend` (porta 3000	Nova rota /eleicoes no React existente
Script Python de importação	`backend` ou script avuls	Executado via child_process.exec ou container temporário

7.2 Variáveis de ambiente

Nenhuma variável nova é obrigatória. O backend já tem DATABASE_URL e REDIS_URL disponíveis. Opcionalmente, adicionar ao .env:

```
# Opcional — controle do módulo eleitoral
TSE_IMPORT_TIMEOUT_MS=600000   # 10 minutos máximo por job
TSE_MAX_JOBS_CONCURRENT=2      # máx jobs simultâneos
TSE_CACHE_MUNICIPIO=true       # compartilhar dados brutos entre tenants
```

7.3 Migrations

Criar migration no sistema de migrations já existente do backend (Prisma, Knex ou migrate conforme o projeto). A migration deve:

15. Criar tabelas tse_candidatos, tse_locais_votacao, tse_votos_secao

16. Criar view materializada votos_por_bairro com índices
17. Adicionar coluna tse_candidato_id na tabela tenants (FK opcional para o candidato principal)
18. Criar índices compostos para queries de frontend (tenant_id + ano_eleicao + nm_bairro)

8. Requisitos Não-Funcionais

Requisito	Meta	Como atingir
Carregamento inicial /eleicoes	< 2 segundos	View materializada + índices compostos
Troca de aba	< 500ms	Dados pré-carregados em estado React
Job de importação	< 10 minutos	Fila Redis + worker assíncrono
Isolamento multi-tenant	100%	WHERE tenant_id = $1 em todas as queries
Mobile (375px)	100% navegável	Scroll horizontal em tabelas + cards empilhados
Sem downtime no deploy	Zero	Migrations não-destrutivas + feature flag

© 2025 VereadorCRM

9. Critérios de Aceite — MVP

Funcionalidade	Descrição	Prioridade
Migration aplicada	Tabelas tse_* e view votos_por_bairro existem no banco sem erros	**Must Have**
Wizard funcional	Configuração salva candidato, dispara job e exibe progresso em tempo real	**Must Have**
Importação Apucarana	Dados de 2024 e 2020 de Apucarana importados corretamente via script Python	**Must Have**
Aba Resumo	7 cards com dados corretos; seletor de eleição funciona sem reload	**Must Have**
Aba Bairros	Tabela ordena, filtra, expande para escolas; dados batem com CSV original do TSE	**Must Have**
Aba Escolas	Top 10 escolas com dados corretos; 'Ver no Maps' abre URL correta	**Must Have**
Aba Comparativo	Delta de votos correto entre 2020 e 2024 por bairro	**Must Have**
Isolamento tenant	Tenant A não acessa dados de tenant B via API (teste com JWT de outro tenant)	**Must Have**
Mobile OK	Todas as abas navegáveis em 375px sem overflow ou sobreposição	**Must Have**
Job não bloqueia API	Importação em background não degrada tempo de resposta dos outros endpoints	**Must Have**

© 2025 VereadorCRM

10. Fora do Escopo — v1.0

- Heatmap cartográfico com mapa geográfico interativo
- Atualização em tempo real durante eleições (dados TSE são estáticos por eleição)
- Comparação automática com todos os concorrentes do município
- Envio de WhatsApp segmentado por bairro via Evolution API (planejado para v1.2)
- Dados de pesquisa de intenção de voto ou surveys
- Análise de financiamento de campanha ou prestação de contas
- Novo container Docker ou serviço externo

© 2025 VereadorCRM

11. Roadmap

Fase	Entregáveis	Estimativa
v0.1 — Base de dados	Migration + script Python de importação + teste com Apucarana 2024	Semana 1
v0.2 — API	Todos os endpoints /api/eleicoes + worker Redis funcionando	Semana 2
v0.3 — Frontend I	Aba Resumo + Aba Bairros + wizard de onboarding	Semana 3
v1.0 — MVP completo	Abas Escolas, Comparativo e Insights + mobile polish + critérios de aceite	Semana 4-5
v1.1	Exportar CSV + cruzamento com constituintes do CRM	Mês 2
v1.2	Envio WhatsApp por bairro via Evolution API + comparar concorrentes	Mês 3

Próximo passo imediato

Criar a migration com as 3 tabelas tse_* e rodar o script Python com dados reais de Apucarana/PR (2024 e 2020) antes de desenvolver qualquer endpoint. Validar que os dados chegam corretos no banco é pré-requisito para tudo o mais.

© 2025 VereadorCRM